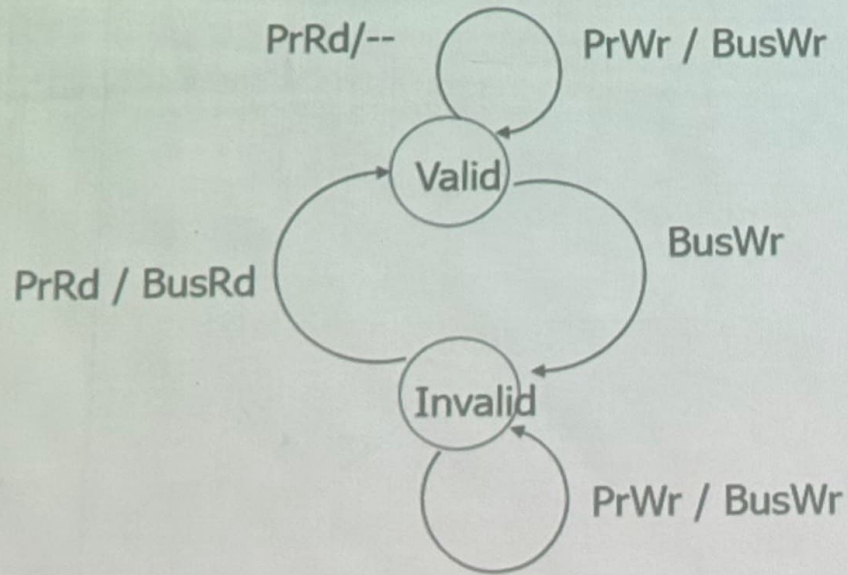
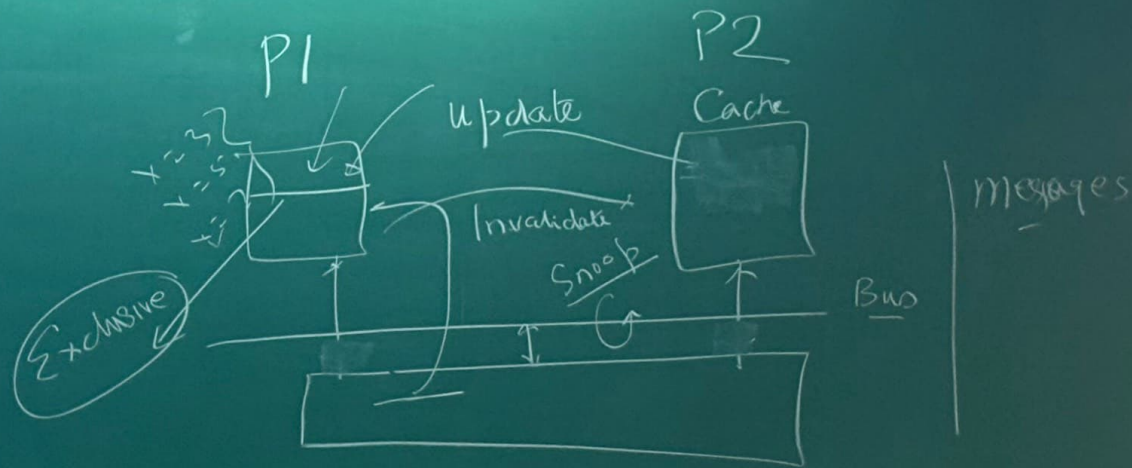


A Simple Snoopy Cache Coherence Protocol

- Caches “snoop” (observe) each others’ write/read operations
- A simple protocol (VI protocol):



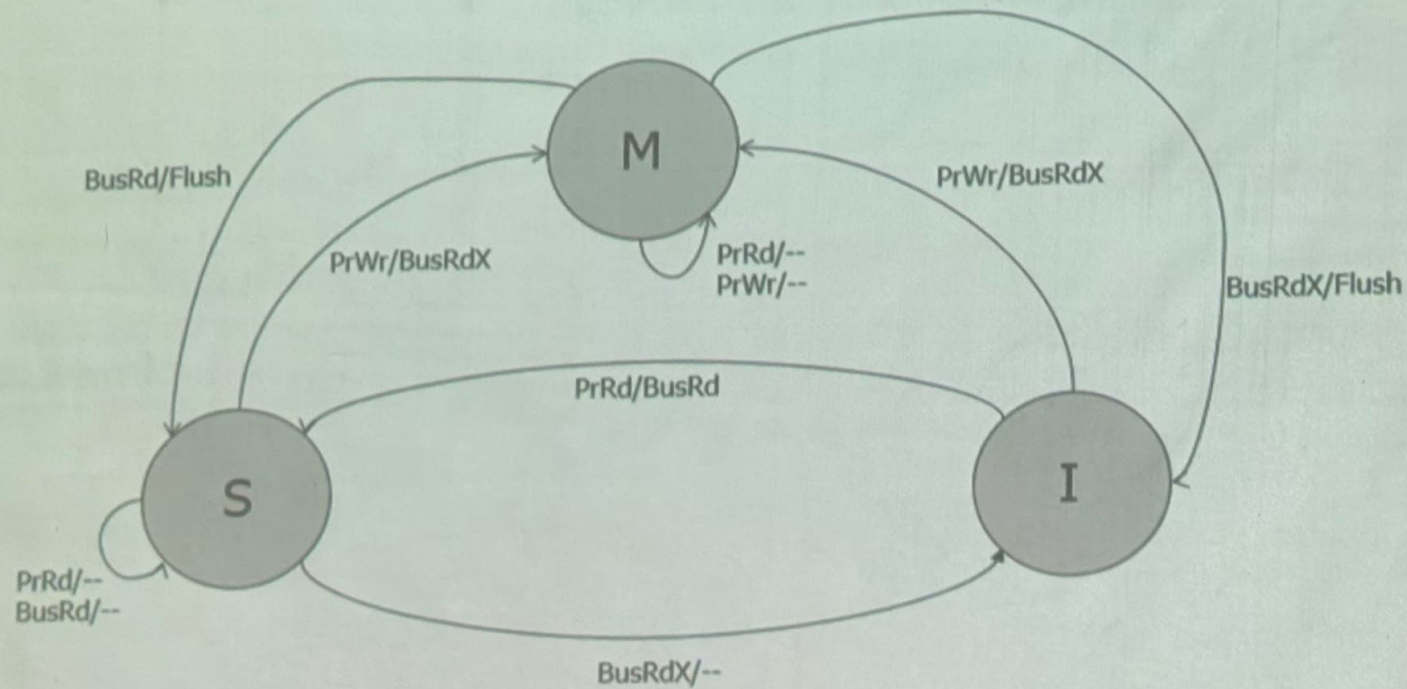
- **Write-through, no-write-allocate** cache
- Actions of the local processor on the cache block: **PrRd**, **PrWr**,
- Actions that are broadcast on the bus for the block: **BusRd**, **BusWr**



A More Sophisticated Protocol: MSI

- Extend metadata per block to encode three states:
 - **M**(odified): cache line is the only cached copy and is dirty
 - **S**(hared): cache line is one of potentially several cached copies and it is clean (i.e., at least one clean cached copy)
 - **I**(nvalid): cache line is not present in this cache
- Read miss makes a *Read* request on bus, transitions to **S**
- Write miss makes a *ReadEx* request, transitions to **M** state
- When a processor snoops *ReadEx* from another writer, it must invalidate its own copy (if any)
- $S \rightarrow M$ upgrade can be made without re-reading data from memory (via Invalidations)

MSI State Machine



ObservedEvent/Action

[Culler/Singh96]

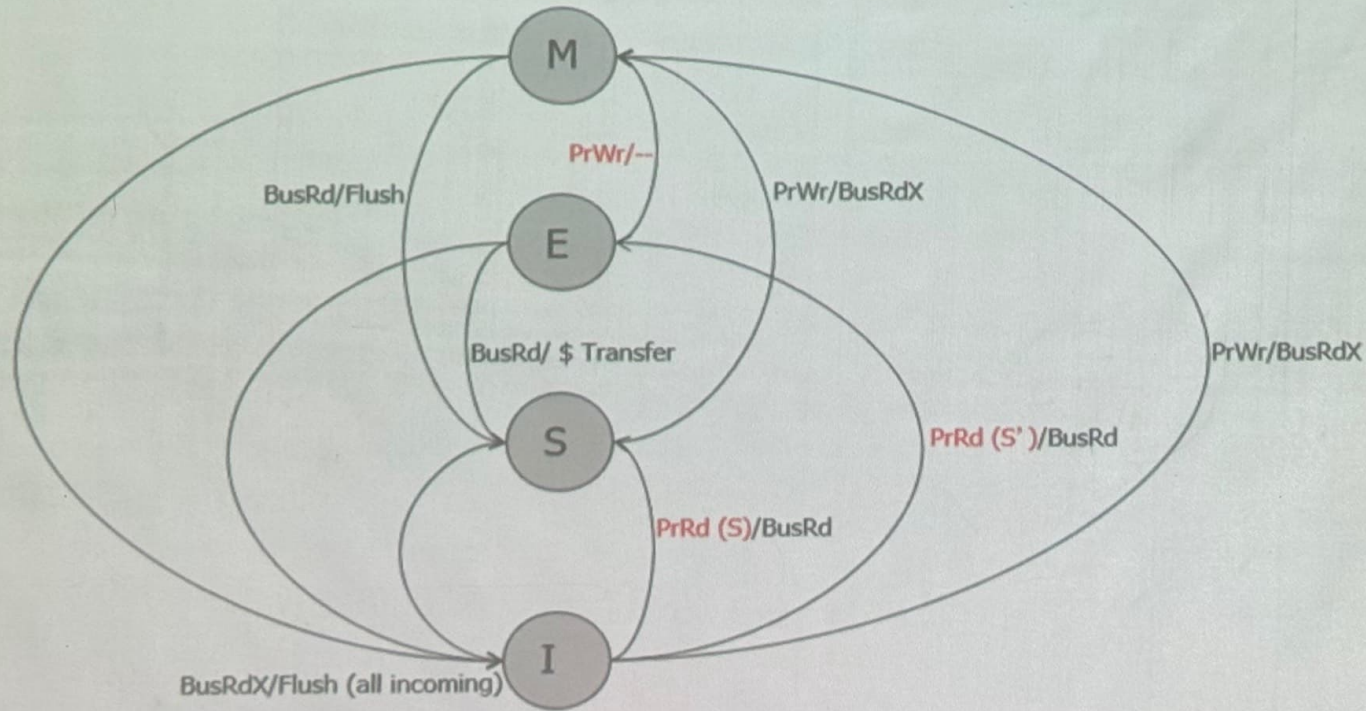
The Problem with MSI

- A block is in no cache to begin with
- Problem: On a read, the block immediately goes to "Shared" state although it may be the only copy to be cached (i.e., no other processor will cache it)
- Why is this a problem?
 - Suppose the cache that reads the block wants to write to it at some point
 - It needs to broadcast "invalidate" even though it has the only cached copy!
 - If the cache knew it had the only cached copy in the system, it could have written to the block without notifying any other cache → saves unnecessary broadcasts of invalidations

The Solution: MESI

- Idea: Add another state indicating that this is the only cached copy and it is clean.
 - *Exclusive state*
- Block is placed into the exclusive state if, during *BusRd*, no other cache had it
 - Wired-OR “shared” signal on bus can determine this: snooping caches assert the signal if they also have a copy
- Silent transition *Exclusive* $\rightarrow$ *Modified* is possible on write!
- MESI is also called the *Illinois protocol*
 - Papamarcos and Patel, “A low-overhead coherence solution for multiprocessors with private cache memories,” ISCA 1984.

MESI State Machine



[Culler/Singh96]

Snoopy Invalidation Tradeoffs

- Should a downgrade from M go to S or I?
 - S: if data is likely to be reused (before it is written to by another processor)
 - I: if data is likely to be not reused (before it is written to by another)
- Cache-to-cache transfer
 - On a BusRd, should data come from another cache or memory?
 - Another cache
 - May be faster, if memory is slow or highly contended
 - Memory
 - Simpler: no need to wait to see if another cache has the data first
 - Less contention at the other caches
 - Requires writeback on M downgrade
- Writeback on Modified->Shared: necessary?
 - One possibility: **Owner** (O) state (MOESI protocol)
 - One cache owns the latest data (memory is not updated)
 - Memory writeback happens when all caches evict copies

P_1 & P_2

Invalidate
Coherence
Protocol

Normal

Bus Tx

Bus Rd

Bus Wr

Bus Rdx

Bus ~~Wrx~~

$X \rightarrow$ main M

Each Cache $\rightarrow$ I

P_1 Read X
 P_2 Read X
 P_1 Write X
 P_3 Write X

MSI $\rightarrow$ 4
MESI $\rightarrow$ 3
MOESI $\rightarrow$ 3

S E

List the Sequence of msg
msl

As 3 Protocols

operation	Bus Action	(P1 P2) Resulting State
P1 does Rd	to Rd X	(S, I)

Difficulties of Multiprocessing

- Much of **parallel computer architecture** is about
 - Designing machines that overcome the sequential and parallel bottlenecks to achieve higher performance and efficiency
 - Making programmer's job easier in writing correct and high-performance parallel programs

Ordering of Operations

- Operations: A, B, C, D
 - In what order should the hardware execute (and report the results of) these operations?
- A contract between programmer and microarchitect
 - Specified by the ISA
- Preserving an “expected” (more accurately, “agreed upon”) order simplifies programmer’s life
 - Ease of debugging; ease of state recovery, exception handling
- Preserving an “expected” order usually makes the hardware designer’s life difficult
 - Especially if the goal is to design a high performance processor: Recall load-store queues in out of order execution and their complexity

Memory Ordering in a Single Processor

- Specified by the von Neumann model
- Sequential order
 - Hardware **executes** the load and store operations **in the order specified by the sequential program**
- Out-of-order execution does not change the semantics
 - Hardware **retires (reports to software the results of)** the load and store operations **in the order specified by the sequential program**
- Advantages: 1) Architectural state is precise within an execution. 2) Architectural state is consistent across different runs of the program → Easier to debug programs
- Disadvantage: Preserving order adds overhead, reduces performance, increases complexity, reduces scalability

Memory Ordering in a Dataflow Processor

- A memory operation executes when its operands are ready
- Ordering specified only by data dependencies
- Two operations can be executed and retired in any order if they have no dependency
- Advantage: Lots of parallelism → high performance
- Disadvantages:
 - Precise state is very hard to maintain (No specified order) → Very hard to debug
 - Order can change across runs of the same program → Very hard to debug

Memory Ordering in a MIMD Processor

- Each processor's memory operations are in sequential order with respect to the "thread" running on that processor (assume each processor obeys the von Neumann model)
- Multiple processors execute memory operations concurrently
- How does the memory see the order of operations from all processors?

Why Does This Even Matter?

- Ease of debugging
 - It is nice to have the same execution done at different times to have the same order of execution → Repeatability
- Correctness
 - Can we have incorrect execution if the order of memory operations is different from the point of view of different processors?
- Performance and overhead
 - Enforcing a strict “sequential ordering” can make life harder for the hardware designer in implementing performance enhancement techniques (e.g., OoO execution, caches)

Protecting Shared Data

- Threads are not allowed to update shared data concurrently
 - For correctness purposes
- Accesses to shared data are encapsulated inside *critical sections* or protected via *synchronization constructs* (locks, semaphores, condition variables)
- Only one thread can execute a critical section at a given time
 - Mutual exclusion principle

Protecting Shared Data

- Threads are not allowed to update shared data concurrently
 - For correctness purposes
- Accesses to shared data are encapsulated inside *critical sections* or protected via *synchronization constructs* (locks, semaphores, condition variables)
- Only one thread can execute a critical section at a given time
 - Mutual exclusion principle
- A multiprocessor should provide the correct execution of synchronization primitives to enable the programmer to protect shared data

Supporting Mutual Exclusion

- Programmer needs to make sure mutual exclusion (synchronization) is correctly implemented
 - We will assume this
 - But, correct parallel programming is an important topic
 - Reading: Dijkstra, "Cooperating Sequential Processes," 1965.
 - <http://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html>
 - See Dekker's algorithm for mutual exclusion
- Programmer relies on hardware primitives to support correct synchronization
- If hardware primitives are not correct (or unpredictable), programmer's life is tough
- If hardware primitives are correct but not easy to reason about or use, programmer's life is still tough

The Problem

- The two processors did **NOT** see the same order of operations to memory
- The “happened before” relationship between multiple updates to memory was inconsistent between the two processors’ points of view
- As a result, each processor thought the other was **not** in the critical section

How Can We Solve The Problem?

- Idea: Sequential consistency
- All processors see the same order of operations to memory
- i.e., all memory operations happen in an order (called the global total order) that is consistent across all processors
- Assumption: within this global order, each processor's operations appear in sequential order with respect to its own operations.

Sequential Consistency

- Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," IEEE Transactions on Computers, 1979

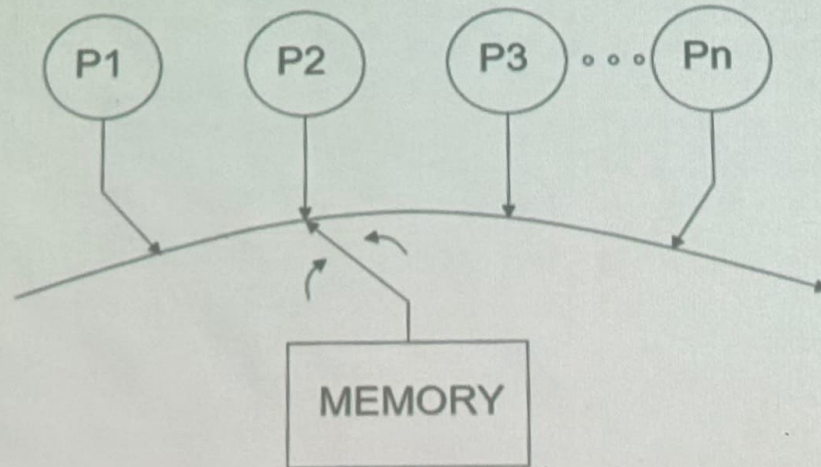
- A multiprocessor system is sequentially consistent if:
- the result of any execution is the same as if the operations of all the processors were executed in some sequential order

AND

- the operations of each individual processor appear in this sequence in the order specified by its program
- This is a memory ordering model, or memory model
- Specified by the ISA

Programmer's Abstraction

- Memory is a switch that services one load or store at a time from any processor
- All processors see the currently serviced load or store at the same time
- Each processor's operations are serviced in program order



Sequentially Consistent Operation Orders

- Potential correct global orders (all are correct):
 - A B X Y
 - A X B Y
 - A X Y B
 - X A B Y
 - X A Y B
 - X Y A B
- Which order (interleaving) is observed depends on implementation and dynamic latencies

Consequences of Sequential Consistency

- Corollaries

1. Within the same execution, all processors see the same global order of operations to memory

→ No correctness issue

→ Satisfies the “happened before” intuition

2. Across different executions, different global orders can be observed (each of which is sequentially consistent)

Issues with Sequential Consistency?

- Nice abstraction for programming, but two issues:
 - Too conservative ordering requirements
 - Limits the aggressiveness of performance enhancement techniques
- Is the total global order requirement too strong?
 - Do we need a global order across all operations and all processors?
 - How about a global order only across all stores?
 - Total store order memory model; unique store order model
 - How about enforcing a global order only at the boundaries of synchronization?
 - Relaxed memory models

Issues with Sequential Consistency?

- Performance enhancement techniques that could make SC implementation difficult
- Out-of-order execution
 - Loads happen out-of-order with respect to each other and with respect to independent stores → makes it difficult for all processors to see the same global order of all memory operations
- Caching
 - A memory location is now present in multiple places
 - Prevents the effect of a store to be seen by other processors → makes it difficult for all processors to see the same global order of all memory operations

Weaker Memory Consistency

- The ordering of operations is important when the order affects operations on shared data → i.e., when processors need to synchronize to execute a “program region”
- Weak consistency
 - Idea: Programmer specifies regions in which memory operations do not need to be ordered
 - “Memory fence” instructions delineate those regions
 - All memory operations before a fence must complete before fence is executed
 - All memory operations after the fence must wait for the fence to complete
 - Fences complete in program order

Tradeoffs: Weaker Consistency

- Advantage
 - No need to guarantee a very strict order of memory operations
 - Enables the hardware implementation of performance enhancement techniques to be **simpler**
 - Can be **higher performance** than stricter ordering
- Disadvantage
 - More **burden on the programmer** or software (need to get the “fences” correct)
- Another example of the programmer-microarchitect tradeoff

TSO

Int $x = y = x1 = x2 = 0$

	<u>T1</u>	Buffer ₁		<u>T2</u>	Buffer ₂
A	$x = 1$		C	$y = 1$	
B	$x1 = y$		D	$x2 = x$	
C/D					

($x1$ $x2$)

nt

1 0

0 1

1 1

0 0

TSO

$d = 2$
 $d = 3$

(0, 0)
(0, 0)

x
m m

Example Question (I)

Question 4 in

<http://www.ece.cmu.edu/~ece447/slides/lib/exe/fetch.php?media=final.pdf>

4. Sequential Consistency [30 points]

Two threads (A and B) are concurrently running on a dual-core processor that implements a *sequentially consistent* memory model. Assume that the value at address 0x1000 is initialized to 0.

Thread A

X1: st 0x1, (0x1000)
X2: ld \$r1, (0x1000)
X3: st 0x2, (0x1000)
X4: ld \$r2, (0x1000)

Thread B

Y1: st 0x3, (0x1000)
Y2: ld \$r3, (0x1000)
Y3: st 0x4, (0x1000)
Y4: ld \$r4, (0x1000)

- (a) List all possible values that can be stored in \$r3 after both threads have finished executing.

Example Question (II)

- (b) After both threads have finished executing, you find that $(\$r1, \$r2, \$r3, \$r4) = (1, 2, 3, 4)$.
How many different *instruction interleavings* of the two threads produce this result?

Example Question (II)

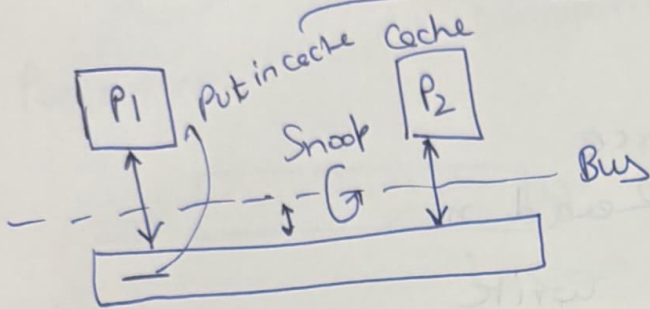
- (b) After both threads have finished executing, you find that $(\$r1, \$r2, \$r3, \$r4) = (1, 2, 3, 4)$. How many different *instruction interleavings* of the two threads produce this result?

- (c) What is the total number of all possible instruction interleavings? You need not expand factorials.

- (d) On a *non-sequentially consistent* processor, is the total number of all possible instruction interleavings less than, equal to, or greater than your answer to question (c)?

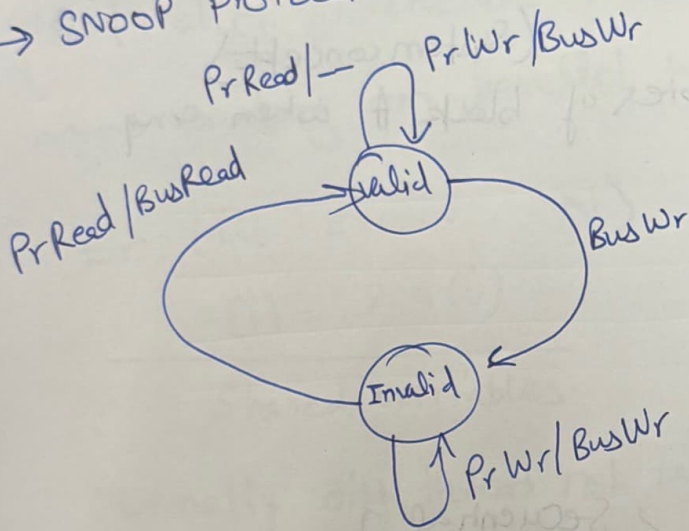
9-oct

- we are interested in All writes to be seen at some time and in Right order



seen by All others, if you update in cache then its duty of processor to update others caches that share data

→ SNOOP protocol

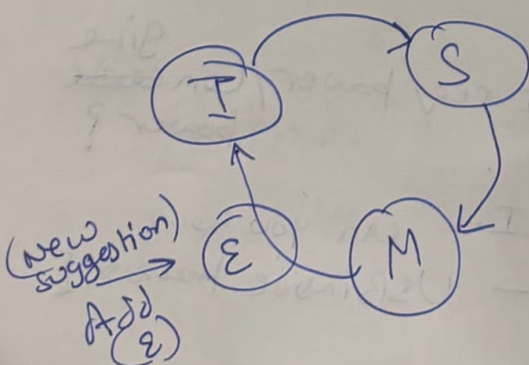


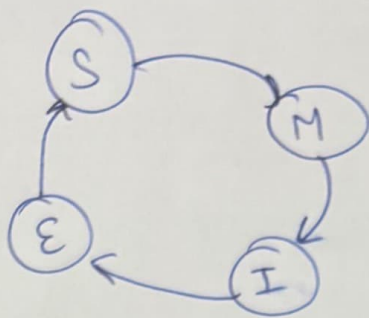
(Slide)

Performance we need to measure
↳ How many messages are being sent

we can introduce one more state in above and that is "Exclusive" - so long we don't share a value with anyone else we need not change state

I = invalid
S = shared
M = modified





Three variations:

MSI
MESI
MOESI

P ₁	P ₂
I → E	I → S
E → M	

Invalidation
S → I Ready
I → S
S → M

Invalidation
M → I Ready
I → S

We assume X is in Memory

Each cache → I

2 processor P₁ & P₂

Invalidation
Coherence
Protocol

P₁ - Read X
P₂ - Read X
P₁ - Write X
P₃ - Write X

Normal Bus Transactions

~ Bus Read
 Bus wr
 Bus Read X

- on bus only one event can go that's why we have
 Serialization out there

List the Seq. of messages for 3 protocols

	Operation	Action	P ₁ Reg State	P ₂ State
Read x	P ₁ - Does Read x	Rd x	S	I
Read x				
Write x				
Write x				

MSI - 4

MEI - 3

MOESI - 3

~ count No. of message for 3 protocol (Imp)

x Read - 9
 x Read - 9
 x Write - 9
 x Write - 9

Invalidate
 (Private)
 Protocol

Difficulties in Multiprocessing

- Its very difficult to figure out where Data Races happen
- Data Races are Non deterministic
- You can not write code to eliminate All data Races

Example of Drive on Read

- "we assume ~~a~~ no one wants to die, p.s, everyone wants to live"
- "Students in class are supposed to leave one row at a time" but if we do that throughput will reduce
- If we have to ensure Data Race do not happen, Program will run correctly but will be too slow

→ we can only reason sequentializing

Application ran on single processor → Result
↑
visible in memory
n - numbers - must be written in that order

in prog we do:-

R₁
R₂
w₁
w₂
w₃
R₃
R₄
w₅

order in prog.
should execute

must be updated in same
exact order

M
E
M
O
R
Y

(Must ensure to get
correctness)

COO is not parallel

→ Reordering itself causes problems

but can do in a way correctness is not violated

Now Imagine Re-ordering in distributed sense

↳ (many more problems come)

→ Memory Model

↳ Across entire memory

* Many processor working on many locations

— All memory locations must be Coherent

Memory Model	Consistent

Sequential order

How test and set ^(mutual exclusion work) Runs?
Does it need hardware support?
Why and why not?

(16)

Problem: two processor may/may not see same order of operations in memory

— Across Diff Runs we have Diff Contentions.

T_1

$x=1$

$r_1=y$

T_2

$y=1$

$r_2=x$

$x=y=r_1=r_2=0$

Assuming seq. (no reorder)

we get 6 possible orders

A	B	C	D
A	D	D	C
⋮			
C	D	A	B

} Possible value

01
10
11

with reorder

0,0

may also come

(Relaxation)

Ex
Total Store Ordering
(Intel)

Total store ordering (Intel)
→ writes you have Prog order
reads you do in any order

- Global order only where you have Synchronisation
- All writes are consistent
- Allow programmer to define Syncro.

Sequential — All in order

Total order — only Store in order

Memory fences — only fences in order

TSO (# of Type of problem in Major)

Every thread has its own buffer (write buffer)

T_1 x set to 1

T_2

$y=1$ set in write buffer

Can potentially see/set

$(x, y = 0, 0)$

Notable to see

for $d=2, d=3$

→ Able to see (0,0)

d = No. of cycles to flush from write buffer

ARM, x86, etc All have diff Memory model

Diff model $\rightarrow$ Diff set of possible answer

if you assume inorder — All have same Ans.